

# Big Data Final Project Report

**Team 22** Predicting US Domestic Flight Arrival Delays Using a Distributed Big Data Pipeline

---

## 1. Title page

- Course: Introduction to Big Data
  - Project: Flight Delay Prediction
  - Team 22 members:
    - Artem Panov (Stage 1, Stage 2 infrastructure, repo structure)
    - Arthur Babkin (Stage 2 EDA, feature engineering, Stage 4 dashboard)
    - Anastasiia Ozerova (Stage 3 ML modelling, Stage 4 Hive layer over ML outputs)
    - Alexander Malyy (technical writer, presentation, repo and reproducibility, pipeline cleanup)
  - Repository: <https://github.com/n4rly-boop/ItoBD-project>
  - Cluster username: team22
-

## 2. Introduction

Flight delays cost airlines and passengers through missed connections and broken plans. Even short delays then lead to late-aircraft penalties and crew problems. The aviation industry uses on-time performance as a main service-quality metric. Predicting delays before they happen helps scheduling and gate assignment.

### Business objectives

1. Identify the operational and temporal patterns that drive arrival delays in US domestic flights.
2. Quantify how delay rates vary across carriers, routes, airports, departure hours, and seasons.
3. Build a binary classifier that predicts whether a given flight will arrive more than 15 minutes late, using only information known before departure.
4. Run the analysis end-to-end on a distributed Big Data stack so the same pipeline scales to multiple years of flight data.

### Problem statement

Given a flight described by its scheduled times, route, carrier, and date, predict the binary target `is_arrival_delayed`, defined as `arr_delay > 15` minutes. The prediction is made before departure, so only pre-departure features are usable: scheduled times, route, carrier, distance, day of week, month, and engineered cyclical and weekend indicators. Cancelled and diverted flights are excluded since their arrival delay is undefined.

---

### 3. Data Description

#### Source

The dataset is the 2024 US domestic flight performance file published on Kaggle (hrishitpatil/flight-data-2024), derived from the Bureau of Transportation Statistics (BTS) On-Time Performance database.

#### Volume

- 7,079,081 rows in the raw load and in the final Hive feature table (12 monthly partitions, 16 carrier buckets). Cancelled and diverted flights (113,814 rows) are kept in the feature table because q4 of Stage 2 (cancellation insight) needs them. Stage 3 filters them out at the start of `model.py`, leaving 6,965,267 rows for model training.
- 36 columns in the raw schema, 40 in the final feature table (4 engineered columns added).
- 1.31 GB on disk as CSV; ~250 MB after Parquet+Snappy compression in Hive.

#### Schema (raw)

The dataset contains:

- Identifiers: `flight_id`, `op_unique_carrier`, `op_carrier_fl_num`
- Temporal features: `year`, `month`, `day_of_month`, `day_of_week`, `fl_date`
- Origin/destination geospatial features: `origin`, `origin_city_name`, `origin_state_nm`, `dest`, `dest_city_name`, `dest_state_nm`, `distance`
- Scheduled times: `crs_dep_time`, `crs_arr_time`, `crs_elapsed_time`
- Actual times: `dep_time`, `arr_time`, `wheels_off`, `wheels_on`, `taxi_in`, `taxi_out`, `actual_elapsed_time`, `air_time`
- Delay outcomes: `dep_delay`, `arr_delay`, `cancelled`, `cancellation_code`, `diverted`
- Delay reason breakdown: `carrier_delay`, `weather_delay`, `nas_delay`, `security_delay`, `late_aircraft_delay`

#### Class balance of the target

On the modelling subset (after Stage 3 filters out 113,814 cancelled and diverted flights), the binary label `is_arrival_delayed = (arr_delay > 15)` splits as:

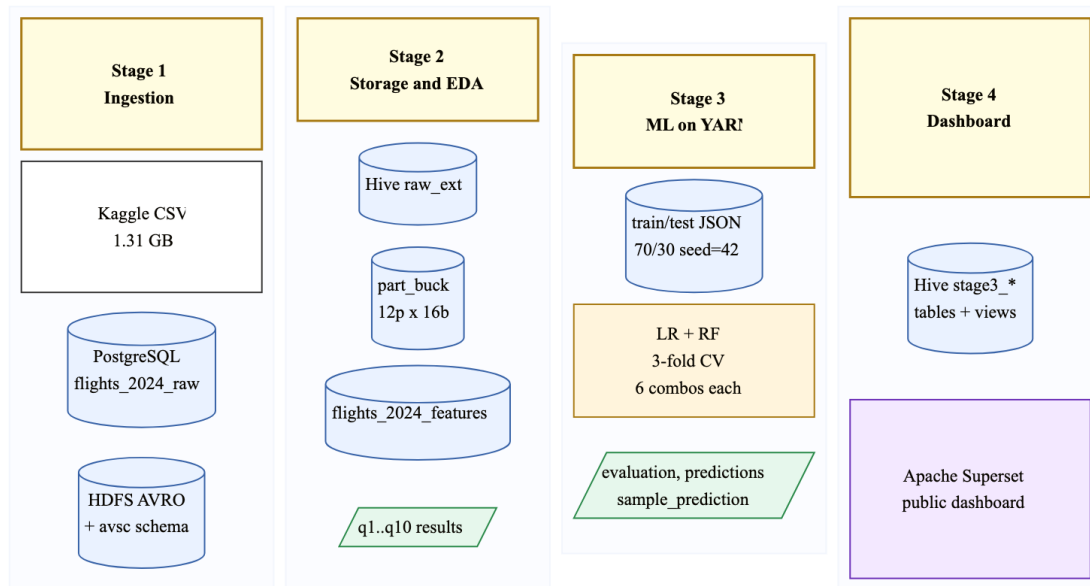
| label | count     | share  |
|-------|-----------|--------|
| 0     | 5,561,879 | 79.85% |
| 1     | 1,403,388 | 20.15% |

About 4:1 imbalance. We handle it in Stage 3 with balanced class weights.

---

## 4. Architecture of the data pipeline

The pipeline runs end-to-end on the Innopolis Hadoop cluster. All stages are reproducible from `main.sh`.



### Pipeline architecture

### Inputs and outputs by stage

| Stage | Input                                  | Output                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Kaggle CSV (data/flight_data_2024.csv) | Postgres table flights_2024_raw in team22_projectdb; AVRO+Snappy files in HDFS project/warehouse/flights_2024_raw/; AVSC + Java schema in output/ (also AVSC copied to HDFS project/warehouse/avsc/)                                                                                                                                                                                                                                                   |
| 2     | HDFS AVRO files + AVSC schema          | Final Hive table flights_2024_features (Parquet+Snappy, partitioned by month, bucketed by op_unique_carrier into 16 buckets); intermediate tables flights_2024_raw_ext and flights_2024_part_buck are dropped during the build; 10 EDA result tables q1_results..q10_results; 10 CSV files output/q1.csv..q10.csv (exported by stage2.sh) and 10 JPG charts output/q1.jpg..q10.jpg (exported manually from Superset); beeline logs output/stage2_*.txt |
| 3     | flights_2024_features Hive table       | Train/test JSON splits in HDFS project/data/{train,test}/; two trained Spark ML pipelines in HDFS project/models/{model1,model2}/;                                                                                                                                                                                                                                                                                                                     |

| Stage | Input                                                  | Output                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |                                                        | per-model predictions in HDFS<br>project/output/{model1,model2}_predictions/<br>evaluation in HDFS project/output/evaluation/<br>sample prediction in output/sample_prediction.csv; 4 charts<br>generated by scripts/charts_stage3.py from<br>evaluation.csv: output/stage3_model_metrics.jpg,<br>output/stage3_training_time.jpg,<br>output/stage3_confusion_matrix_model1.jpg,<br>output/stage3_confusion_matrix_model2.jpg; logs<br>output/stage3_model.log, output/stage3_charts.txt,<br>output/stage3_pylint.txt                                                                                                                                                                                                                                                                                                  |
| 4     | Stage 3 HDFS<br>outputs;<br>Stage 2 & 3 chart<br>files | Two summary CSVs output/stage3_feature_summary.csv and<br>output/stage3_hyperparameter_summary.csv (also uploaded<br>to HDFS); 6 external Hive tables (stage3_evaluation,<br>stage3_model1_predictions, stage3_model2_predictions,<br>stage3_sample_prediction, stage3_feature_summary,<br>stage3_hyperparameter_summary) and 5 views<br>(stage3_model_metrics_long,<br>stage3_confusion_matrix_long, stage3_prediction_counts,<br>stage3_hyperparameter_optimization,<br>stage3_model_comparison) created from<br>sql/create_stage3_hive_tables.hql; beeline logs<br>output/create_stage3_hive_tables.txt and<br>output/check_stage3_hive_tables.txt; published Apache<br>Superset dashboard (built manually on top of these Hive tables,<br>plus the 10 EDA charts from Stage 2 and the 4 ML charts from<br>Stage 3) |

## 5. Data Preparation

### PostgreSQL ingestion

scripts/build\_projectdb.py connects to team22\_projectdb on hadoop-04, executes sql/create\_tables.sql to build the relational schema, then bulk-loads data/flight\_data\_2024.csv via COPY (psycopg2 copy\_expert). Validation queries from sql/test\_database.sql confirm row counts and basic constraints.

The relational design is kept simple: a single fact table flights\_2024\_raw with 36 columns that mirror the BTS file. We did not split it into a star schema because the analytical layer is Hive on Parquet, not OLAP on Postgres. Postgres is only the entry point for Sqoop, as required by the assignment.

### ER diagram

The Postgres layer has one table:

| flights_2024_raw |                   |               |                     |               |
|------------------|-------------------|---------------|---------------------|---------------|
| PK               | <u>flight_id</u>  | BIGSERIAL     | wheels_off          | NUMERIC(10,1) |
|                  | year              | INTEGER       | wheels_on           | NUMERIC(10,1) |
|                  | month             | SMALLINT      | taxi_in             | NUMERIC(10,2) |
|                  | day_of_month      | SMALLINT      | crs_arr_time        | INTEGER       |
|                  | day_of_week       | SMALLINT      | arr_time            | NUMERIC(10,1) |
|                  | fl_date           | DATE          | arr_delay           | NUMERIC(10,2) |
|                  | op_unique_carrier | VARCHAR(10)   | cancelled           | SMALLINT      |
|                  | op_carrier_fl_num | NUMERIC(10,1) | cancellation_code   | VARCHAR(1)    |
|                  | origin            | VARCHAR(10)   | diverted            | SMALLINT      |
|                  | origin_city_name  | VARCHAR(128)  | crs_elapsed_time    | NUMERIC(10,2) |
|                  | origin_state_nm   | VARCHAR(128)  | actual_elapsed_time | NUMERIC(10,2) |
|                  | dest              | VARCHAR(10)   | air_time            | NUMERIC(10,2) |
|                  | dest_city_name    | VARCHAR(128)  | distance            | NUMERIC(10,2) |
|                  | dest_state_nm     | VARCHAR(128)  | carrier_delay       | INTEGER       |
|                  | crs_dep_time      | INTEGER       | weather_delay       | INTEGER       |
|                  | dep_time          | NUMERIC(10,1) | nas_delay           | INTEGER       |
|                  | dep_delay         | NUMERIC(10,2) | security_delay      | INTEGER       |
|                  | taxi_out          | NUMERIC(10,2) | late_aircraft_delay | INTEGER       |

The full DDL with NOT NULL and CHECK constraints is in sql/create\_tables.sql.

## Sqoop to HDFS

Stage 1 imports the Postgres table to HDFS via `sqoop import-all-tables` with `--as-avrodatafile` and `--compression-codec=snappy`. The generated `.avsc` schema is uploaded to `project/warehouse/avsc` for Stage 2.

## Hive tables

Three Hive tables are built in sequence by `scripts/stage2.sh`:

1. **flights\_2024\_raw\_ext** (`sql/db.hql`): external table over the AVRO files using the AVSC schema. Used only to feed the optimised table.
2. **flights\_2024\_part\_buck** (`sql/optimize_flights.hql`): external Parquet table with Snappy compression, partitioned by month (12 partitions) and clustered by `op_unique_carrier` into 16 buckets. Built via `INSERT OVERWRITE ... PARTITION (month) SELECT ... FROM flights_2024_raw_ext`. Cancelled flights are kept here so Stage 2 can compute the cancellation-rate insight.
3. **flights\_2024\_features** (`sql/build_features.hql`): the final feature table. Same partition/bucket layout, Parquet+Snappy. Adds four engineered columns:
  - `scheduled_dep_hour = FLOOR(crs_dep_time / 100)`
  - `scheduled_arr_hour = FLOOR(crs_arr_time / 100)`
  - `is_weekend = CASE WHEN day_of_week IN (6,7) THEN 1 ELSE 0 END`
  - `is_arrival_delayed = CASE WHEN arr_delay > 15 THEN 1 ELSE 0 END`

After step 2, `sql/finalize_stage2.hql` drops `flights_2024_raw_ext` (no longer needed) and runs sanity queries on `flights_2024_part_buck` (row count, date range, `DESCRIBE FORMATTED`). Then step 3 (`build_features.hql`) creates `flights_2024_features` and drops `flights_2024_part_buck` at the end. This leaves one main table for the next stages and saves warehouse space. `flights_2024_features` keeps cancelled and diverted rows so that Stage 2 q4 can compute the cancellation-rate insight; Stage 3 filters them out at the start of `model.py`. The full layered design can be rebuilt from the `.hql` files under `sql/`.

## Sample database content

`SELECT * FROM flights_2024_features LIMIT 5` returns a row per flight with all 40 columns including the engineered ones. A representative row:

|                    |        |
|--------------------|--------|
| flight_id          | : 1    |
| month              | : 1    |
| day_of_week        | : 2    |
| op_unique_carrier  | : DL   |
| origin             | : ATL  |
| dest               | : MIA  |
| crs_dep_time       | : 0805 |
| crs_arr_time       | : 1015 |
| distance           | : 594  |
| arr_delay          | : -3   |
| cancelled          | : 0    |
| diverted           | : 0    |
| scheduled_dep_hour | : 8    |
| scheduled_arr_hour | : 10   |
| is_weekend         | : 0    |
| is_arrival_delayed | : 0    |

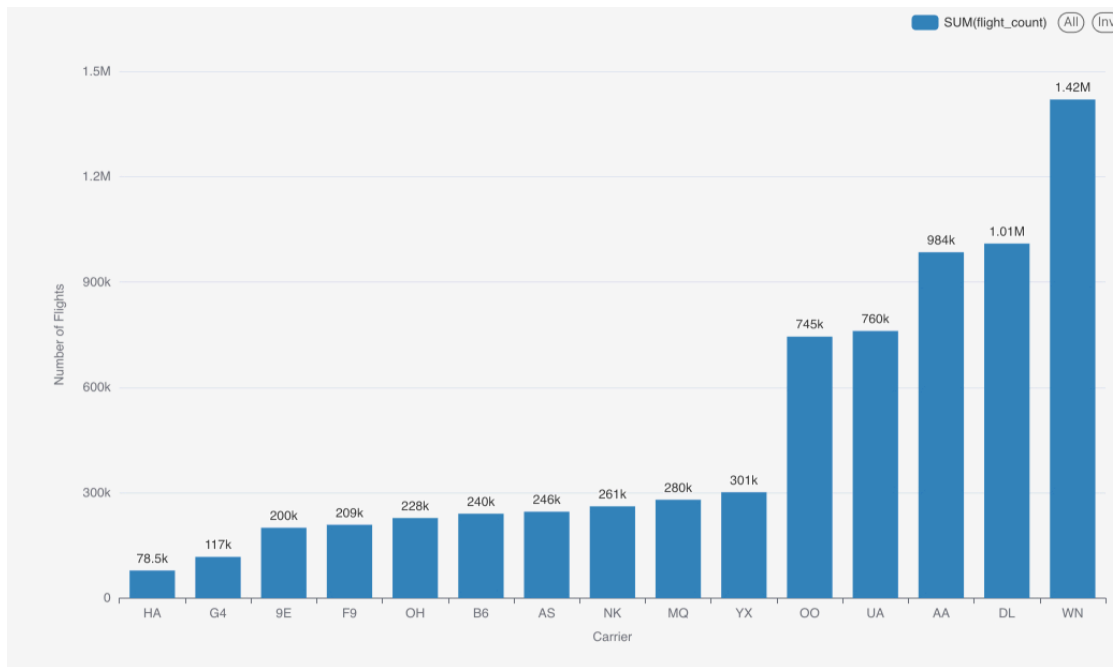
---



## 6. Data Analysis (Stage 2 EDA)

Ten exploratory queries run as `sql/q1.hql` to `sql/q10.hql`. Each produces a Hive result table, a CSV in `output/qX.csv`, and a chart in `output/qX.jpg`.

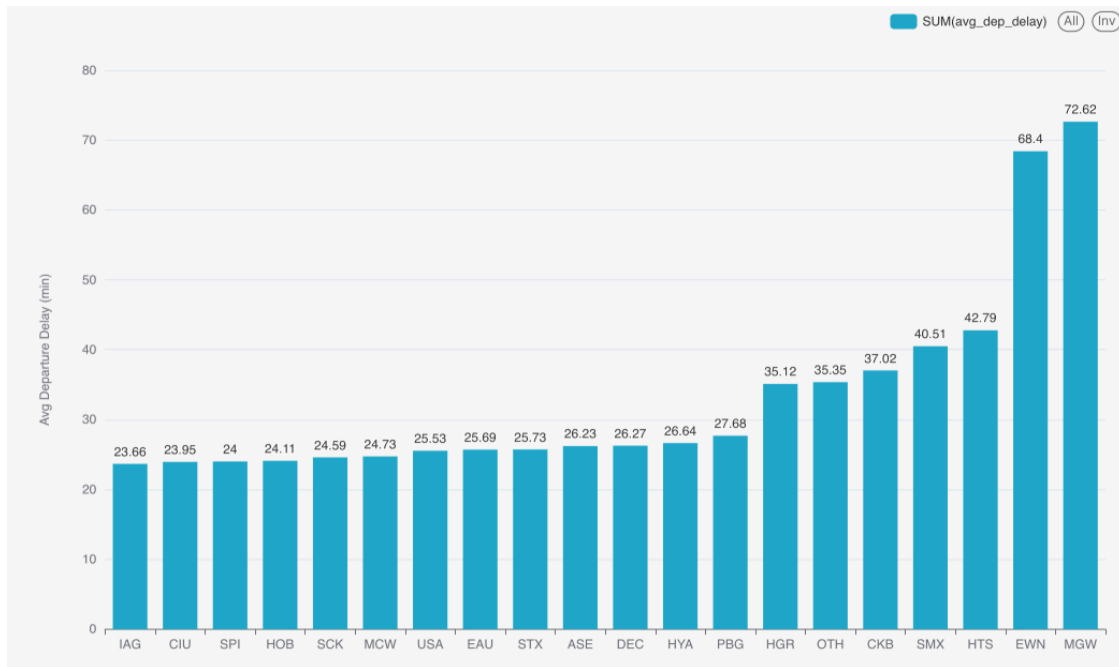
### q1: Flight count by carrier



*q1*

WN, AA, DL, and UA together fly over half of all 2024 domestic flights. Smaller regionals contribute long tails of low-volume routes.

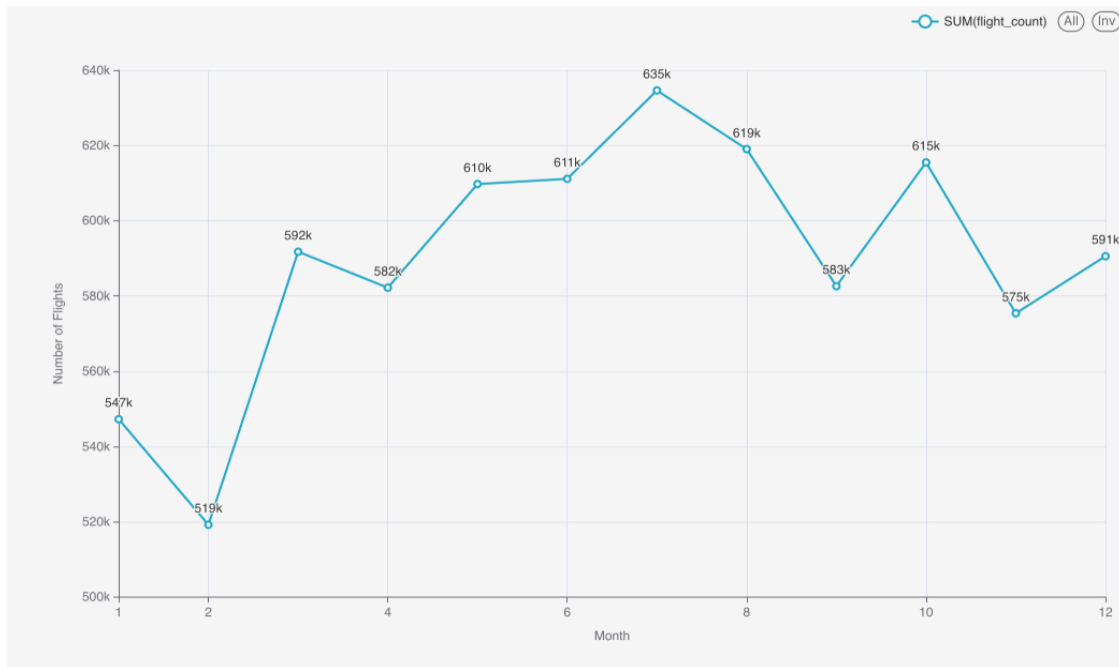
## q2: Top-20 origin airports by average departure delay



q2

Smaller mid-tier airports (PUW, ESC, OWB and similar) have the highest mean departure delay, often because one late aircraft delays the rest of the day's schedule (there is no buffer). Major hubs (ATL, ORD) have a lower mean delay even though they have more delays in total, because their mean is spread over many more flights.

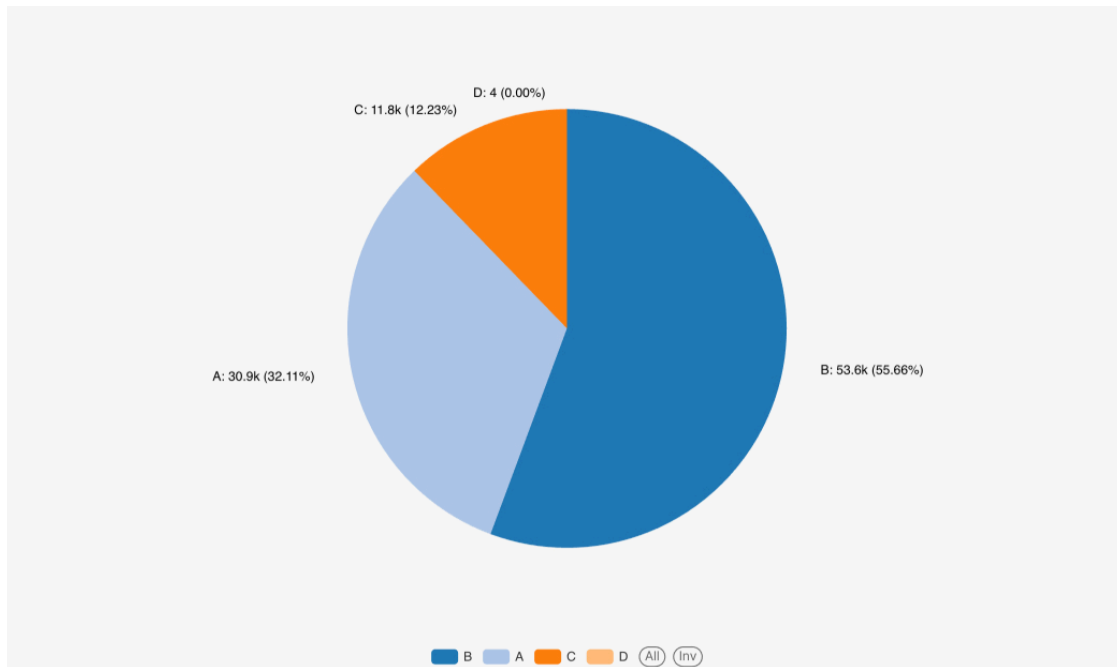
### q3: Monthly flight count (seasonality)



q3

Flight volume peaks in summer (June to August) with a secondary peak around December holidays. February is the lightest month. The seasonal envelope confirms why models include month and its sin/cos cyclical encoding.

#### q4: Cancellation reasons and percentages



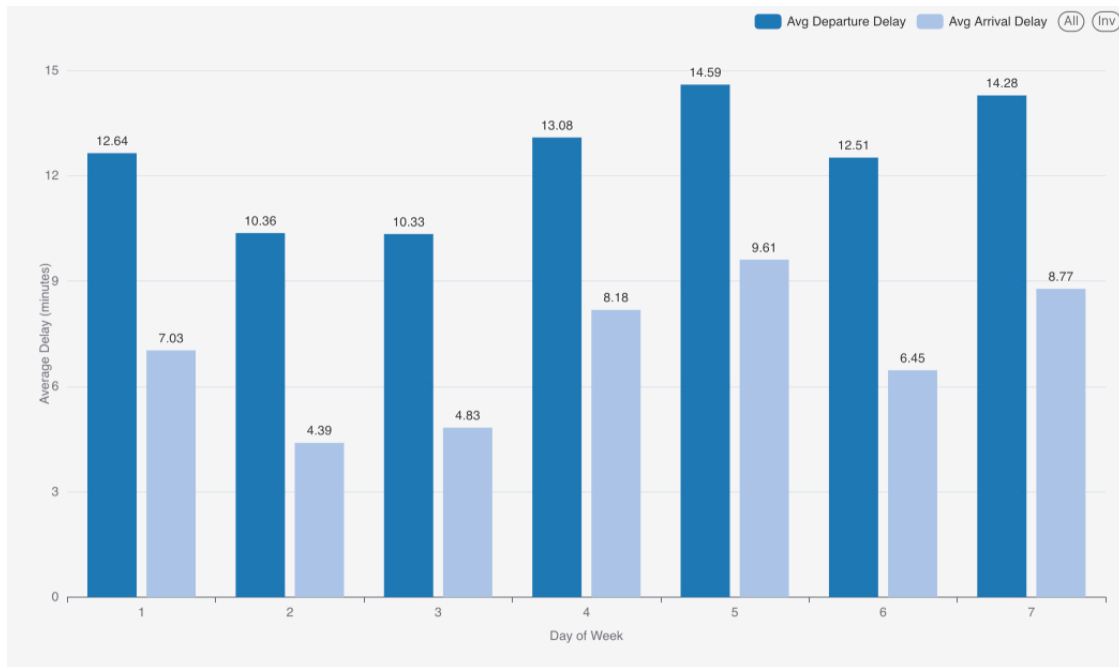
q4

Of cancelled flights:

- B (Weather): largest share
- A (Carrier): second
- C (NAS air-traffic): third
- D (Security): negligible

Because weather drives most cancellations, a weather-aware predictor is a natural follow-up.

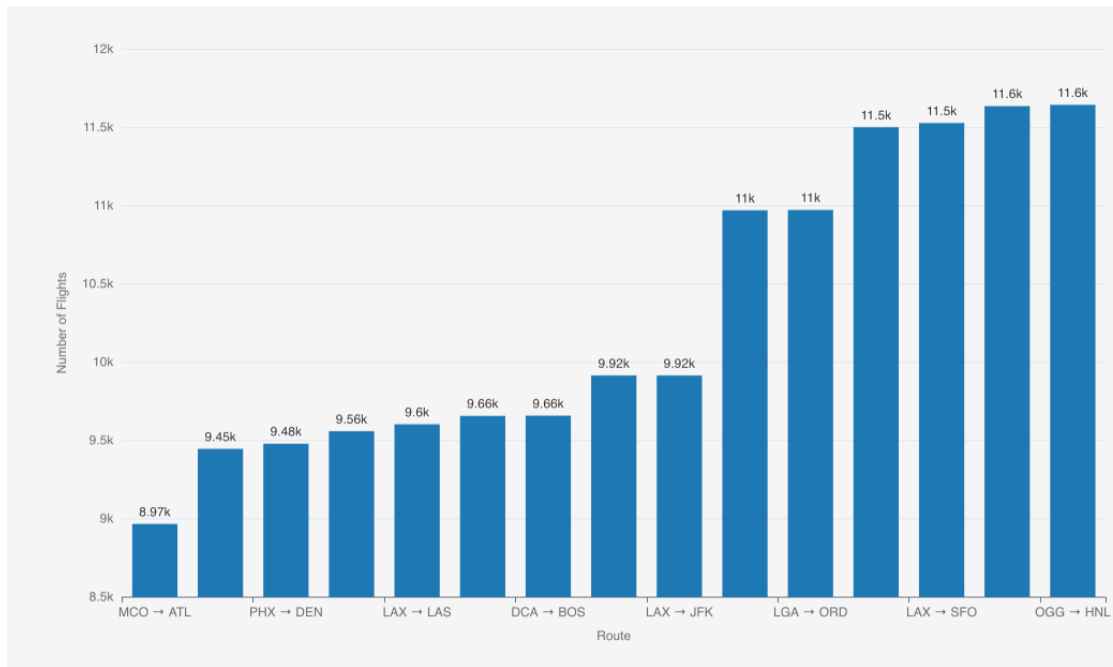
## q5: Punctuality by day of week



q5

Mid-week (Tuesday, Wednesday) is the most punctual; Sunday and Friday show the highest mean arrival delay. The `is_weekend` indicator captures the weekend effect, while `day_of_week` and its sin/cos encoding capture finer mid-week structure.

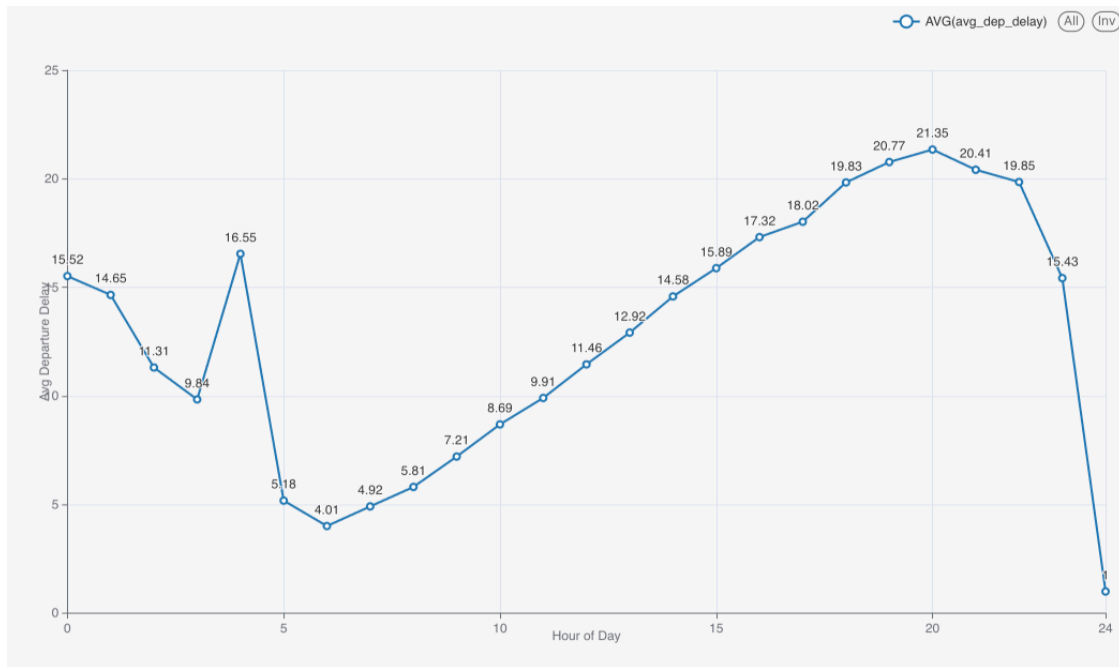
## q6: Top-15 busiest routes



q6

The busiest route is HNL-OGG (Honolulu to Kahului, Hawaii) with about 11,600 flights each way, followed by LAX-SFO (~11,500) and LGA-ORD (~11,000). Two-way traffic is nearly balanced on every route in the top 15. The least busy route on the list, MCO-ATL, still has 8,967 flights.

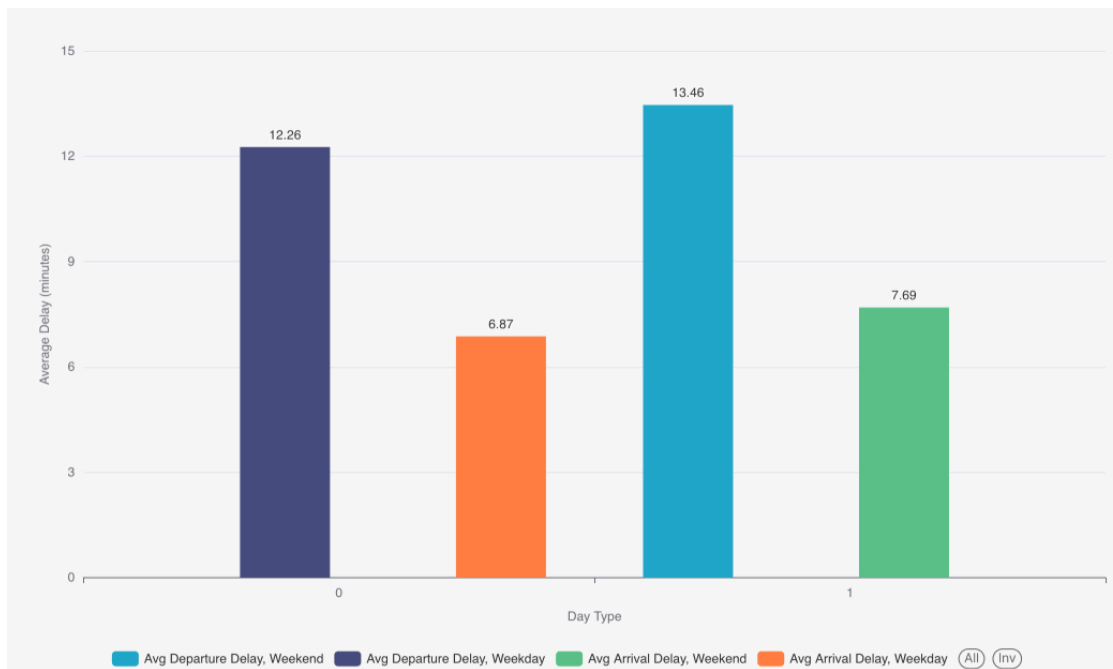
### q7: Average departure delay by scheduled hour



q7

Delays follow a clear daily pattern: lowest in the early morning (5 to 7 AM), growing during the day, highest at 6 to 8 PM, then dropping a bit overnight. This pattern is the reason we use cyclical hour encoding in Stage 3.

## q8: Weekend vs weekday

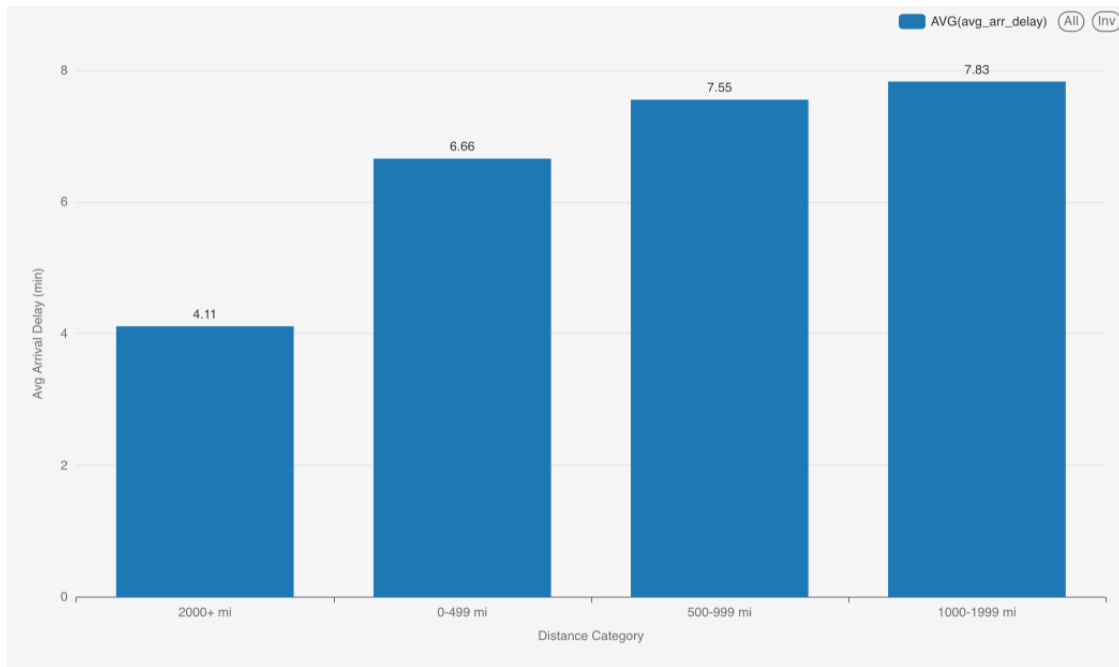


q8

Weekends show mean arrival delay roughly 1 to 2 minutes higher than weekdays. The effect is small but consistent and supports keeping `is_weekend` as a feature.



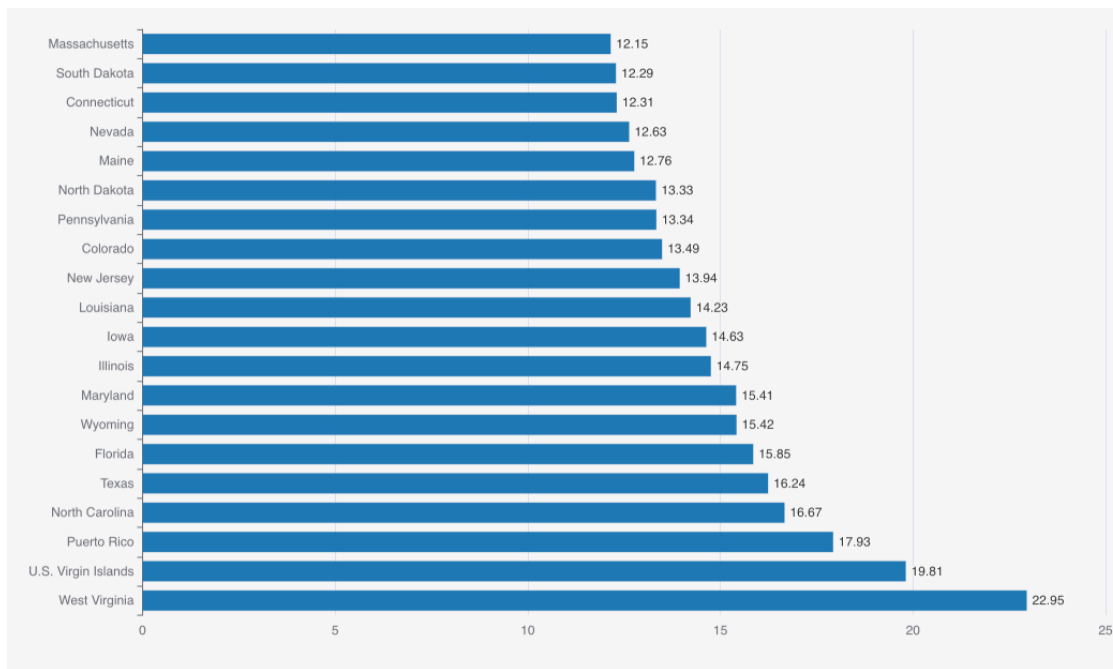
q9: Average arrival delay by distance category



q9

Medium-haul flights (500-1999 miles) have the worst mean arrival delay, around 7.6-7.8 minutes. Short flights (under 500 miles) come second at 6.7 minutes. Long-haul flights (2000+ miles) are the most punctual at only 4.1 minutes, because their scheduled block has more buffer time that hides some en-route delay.

## q10: Top-20 origin states by average departure delay



q10

West Virginia has the worst mean departure delay of all 50 states. U.S. Virgin Islands (~19.8 min) and Puerto Rico are next, likely because of island logistics and weather. Even the best states (Massachusetts, Connecticut at ~12.3 min) sit above 12 minutes. The delay range across states is almost 2x.

---

## 7. ML Modelling (Stage 3)

### Task type and metrics

Binary classification on a 4:1-imbalanced label. The two main metrics for the rubric:

- **Area under ROC** (areaUnderROC): how well the model separates the two classes across all thresholds. Less sensitive to class imbalance than accuracy.
- **Area under PR curve** (areaUnderPR): focused on the positive class (delayed flights). Useful when the positive class is rare.

`scripts/model.py` also reports accuracy, weightedPrecision, weightedRecall, weightedF1 (via `MulticlassClassificationEvaluator`), the 2x2 confusion matrix, and per-class plus macro precision/recall/F1. AUC-ROC and AUC-PR remain the headline numbers because of the 4:1 class imbalance.

### Feature extraction pipeline

All stages run inside a single `pyspark.ml.Pipeline`:

1. `StringIndexer` x 3 on `op_unique_carrier`, `origin`, `dest` with `handleInvalid="keep"`.
2. `OneHotEncoder` on the three indexed columns.
3. `VectorAssembler` combining the OHE vectors, three direct numeric features (`crs_elapsed_time`, `distance`, `is_weekend`), and 10 cyclical sin/cos features encoding `scheduled_dep_hour`, `scheduled_arr_hour`, `day_of_week`, `month`, and `day_of_month`.
4. `StandardScaler` with `withStd=True`, `withMean=False`. If we also centred the data, the OHE block would become dense and use much more memory. So we only scale.

Raw integer time columns (`scheduled_dep_hour`, `day_of_week`, `month`, `day_of_month`) are not passed directly to the model. They are used only as inputs to the cyclical encoding step.

### Cyclical sin/cos encoding

Linear models cannot infer that hour 23 is close to hour 0 from a raw integer column. We add  $\sin(2\pi x/\text{period})$  and  $\cos(2\pi x/\text{period})$  for each cyclical variable: hour (period 24), day-of-week (period 7), month (period 12), day-of-month (period 31). Logistic regression then treats hour 23 and hour 0 as adjacent.

## Class weighting

Inverse-frequency weights are calculated from the train set:

- $w_0 = \text{total} / (2 * \text{count}_0) \approx 0.626$
- $w_1 = \text{total} / (2 * \text{count}_1) \approx 2.480$

Both estimators receive the weights via `weightCol="class_weight"`, so the loss treats positives roughly 4x more strongly than negatives.

## Train/test split

`randomSplit([0.7, 0.3], seed=42)`. Train: 4,875,184 rows. Test: 2,090,083 rows. Splits are saved to HDFS as JSON (project/data/train, project/data/test).

## Model 1: Logistic Regression

Hyperparameter grid (3 x 2 = 6 combinations):

| Param           | Values           |
|-----------------|------------------|
| regParam        | 0.001, 0.01, 0.1 |
| elasticNetParam | 0.0, 0.5         |

Tuned with 3-fold cross-validation (`CrossValidator`, `parallelism=2`, `seed=42`).

## Model 2: Random Forest

Hyperparameter grid (3 x 2 = 6 combinations):

| Param    | Values      |
|----------|-------------|
| numTrees | 20, 50, 100 |
| maxDepth | 5, 10       |

Same 3-fold CV.

These hyperparameters were selected because they directly control model complexity and the bias-variance trade-off. For Logistic Regression, `regParam` controls regularization strength and `elasticNetParam` controls the L1/L2 regularization mix, which is important after `OneHotEncoding` creates a sparse feature space. For Random Forest, `numTrees` controls ensemble stability and training cost, while `maxDepth` controls tree complexity and overfitting risk. The grid was limited to 6 combinations per model to keep 3-fold cross-validation computationally feasible on the shared YARN cluster.

## Results (test set)

| Model                      | areaUnderROC | areaUnderPR | accuracy | weightedF1 |
|----------------------------|--------------|-------------|----------|------------|
| model1 logistic regression | 0.6602       | 0.3217      | 0.6048   | 0.6439     |
| model2 random forest       | 0.6622       | 0.3327      | 0.6058   | 0.6446     |

Random Forest wins on all four metrics by a small margin. Both models are above the 0.5 random baseline. AUC-PR is above the 0.20 prior, so both models give some lift on the positive class.

## Sample prediction

The best model (Random Forest) is applied to a single held-out test row. The output written to `output/sample_prediction.csv` contains the actual label, the predicted label, and the predicted class probabilities. This is the end-to-end inference check required by the rubric.

## Pylint score

`output/stage3_pylint.txt` reports 9.24/10 on `scripts/model.py`.

## Caveats and future work

These numbers are not the ceiling for this dataset. The hyperparameter grid is small (6 combos per model, 3-fold CV), sized to fit the shared YARN queue and the time budget for the assignment. Our main goal in Stage 3 was to get the full Spark ML pipeline working end-to-end on YARN with reproducible writes to HDFS, not to push metrics. Wider grids (more `regParam` and `elasticNetParam` values for logistic regression, more `numTrees` and deeper `maxDepth` for random forest) would probably raise both AUC-ROC and AUC-PR.

Future work has a lot of room:

- bigger grid and 5-fold CV once the queue is free,
  - gradient-boosted trees (`GBTClassifier`) as a third model,
  - target encoding for high-cardinality fields like `origin` and `dest` instead of one-hot,
  - weather features (q4 shows weather drives most cancellations),
  - a regression head for delay in minutes next to the binary classifier.
-

## 8. Data Presentation (Stage 4 dashboard)

**Dashboard URL:** <http://hadoop-03.uni.innopolis.ru:8808/superset/dashboard/225/>

The dashboard has three sections, in order.

### Section 1: Data Overview

- Source (Kaggle Flight Data 2024), total records (7,079,081 flights), feature count (36 original + 4 engineered), target variable, task description.
- Cancelled and diverted flights removed for ML training; rows with missing `arr_delay` or `dep_delay` filtered out for delay-related EDA queries.
- A few rows from `team22_projectdb.flights_2024_features` so the viewer sees what each row looks like.
- All 40 columns of the feature table with one-line description and Hive type.

### Section 2: Exploratory Analysis (Q1..Q10)

One chart per query from §6, each with 3-5 bullet insights drawn from the underlying Hive query result:

- **Q1.** Flight Count by Carrier (bar)
- **Q2.** Top 20 Airports by Avg Departure Delay (bar)
- **Q3.** Number of Flights per Month (bar)
- **Q4.** Cancellation Reasons (share)
- **Q5.** Average Delays by Day of Week (grouped bar, departure + arrival)
- **Q6.** Top 15 Busiest Routes (bar)
- **Q7.** Avg Departure Delay by Hour of Day (line)
- **Q8.** Weekend vs Weekday Delays (grouped bar)
- **Q9.** Avg Arrival Delay by Distance Category (bar)
- **Q10.** Top 20 States by Avg Departure Delay (bar)

### Section 3: Machine Learning

- Feature engineering (cyclical hour, `is_weekend`, `StringIndexer` + `OneHotEncoder`), models (Logistic Regression, Random Forest with class weights), 3-fold CV, 6 grid combinations per model, metrics list (Accuracy, F1, Area Under ROC, Area Under PR).
- Sourced from `stage3_feature_summary`. Counts and descriptions of categorical features (3), cyclical features (10), direct numeric features (3), scaling, class balancing.
- Sourced from `stage3_hyperparameter_summary`. Grid values per model: LR (`regParam` x `elasticNetParam` = 6), RF (`numTrees` x `maxDepth` = 6).
- Bar chart of AUC-ROC and AUC-PR per model from `stage3_evaluation`.

- Counts per (actual\_status, model) using the stage3\_confusion\_matrix\_long view.
  - Bar chart of training wall-clock per model.
  - Predicted-class breakdown for models.
-

## 9. Conclusion

We built a Big Data pipeline that loads 7M+ US domestic flight records into PostgreSQL, moves them to HDFS via Sqoop, stores them in Hive as partitioned and bucketed Parquet, shows ten business insights in Superset, trains two cross-validated Spark ML classifiers, and presents the result in a dashboard. The final feature table `flights_2024_features` has 7,079,081 rows in 12 monthly partitions and 16 carrier buckets. The Random Forest model gets 0.6622 AUC-ROC and 0.3327 AUC-PR on a held-out 30% test set, above the 0.20 imbalanced-baseline PR.

The exploratory analysis showed three interesting facts:

1. Delays follow a daily pattern: early morning is the best time, late afternoon is the worst.
2. Smaller airports have higher mean delays than major hubs because they have less buffer in their schedules.
3. Weather is the main cancellation reason and explains a lot of the difference between states.

Together with the predictive model, these findings let an airline operations team mark likely-late flights before pushback.

---



## 10. Reflections

### Challenges encountered

- **Artem.** Some friction with bash scripting and PostgreSQL schema setup in Stage 1, mostly around getting psychopg2 COPY and `sql/create_tables.sql` to run cleanly end-to-end. Solved after a few iterations.
- **Arthur.** Building the Superset dashboard was the hardest part: chart layout, dataset registration. Communication inside the team also took effort, since the Stage 2 feature table and Stage 4 dashboard had to line up with what Stage 3 produced.
- **Anastasiia.** Hyperparameter selection on a 4:1 imbalanced dataset was hard. Picking sensible ranges for `regParam`, `numTrees`, `maxDepth` without overfitting needed several CV runs. On top of that, cluster connection issues and long waits in the Spark queue (other teams running their own Stage 3 jobs at the same time) slowed iteration a lot.
- **Alexander.** Had troubles with team organization and helping with communication, which led to some misunderstandings around task delegation.

### Recommendations

See §7 *Caveats and future work* for ML-specific extensions (bigger grid, GBT, target encoding, weather features, regression head). One pipeline-level recommendation goes beyond modelling:

- Move from CSV batch inputs to a streaming + batch hybrid (Kafka + Spark Structured Streaming) for same-day forecasts. The current pipeline rebuilds feature tables from a static yearly file; a streaming source would let predictions update as new flights are scheduled.

## Contribution table

Each row sums to 100%. Names: Artem = Artem Panov, Arthur = Arthur Babkin, Anastasiia = Anastasiia Ozerova, Alex = Alexander Malyy.

| Project task                                     | Description                                                                                                      | Artem | Arthur | Anastasiia | Alex | Deliverables                                                                                                             |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-------|--------|------------|------|--------------------------------------------------------------------------------------------------------------------------|
| Team and dataset selection                       | Read assignment, propose datasets, agree on roles, submit Google Sheet                                           | 25%   | 25%    | 25%        | 25%  | dataset choice                                                                                                           |
| Stage 1: Postgres schema + ingestion + Sqoop     | Build relational schema, load Kaggle CSV, Sqoop AVRO+Snappy to HDFS                                              | 100%  | 0%     | 0%         | 0%   | sql/create_tables.sql, sql/import_data.sql, scripts/build_projectdb.py, scripts/stage1.sh, AVSC + Java schema in output/ |
| Stage 2: raw + partitioned/ bucketed Hive tables | External AVRO table, Parquet+Snappy partitioned by month, bucketed by carrier into 16 buckets                    | 100%  | 0%     | 0%         | 0%   | sql/db.hql, sql/optimize_flights.hql, sql/finalize_stage2.hql, initial scripts/stage2.sh                                 |
| Stage 2: feature engineering                     | Engineered columns (scheduled_dep_hour, scheduled_arr_hour, is_weekend, is_arrival_delayed), final feature table | 0%    | 100%   | 0%         | 0%   | sql/build_features.hql, flights_2024_features table, stage2.sh step [6/6]                                                |

| Project task                             | Description                                                                                                                                | Artem | Arthur | Anastasiia | Alex | Deliverables                                                                                                                                    |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|-------|--------|------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Stage 2: 10 EDA insights                 | 10 HiveQL queries, CSV exports, Superset chart exports                                                                                     | 0%    | 100%   | 0%         | 0%   | sql/q1.hql-q10.hql, output/q1.csv-q10.csv, output/q1.jpg-q10.jpg                                                                                |
| Stage 3: Spark ML pipeline               | Feature extraction pipeline, LR + RF, 6-combo grid each, 3-fold CV, class weights, sample prediction, training on YARN                     | 0%    | 0%     | 100%       | 0%   | scripts/model.py, scripts/stage3.sh, output/evaluation.csv, output/model{1,2}_predictions.csv, output/sample_prediction.csv, models/model{1,2}/ |
| Stage 4: Hive layer over Stage 3 outputs | External tables and views over Stage 3 HDFS results, summary CSVs                                                                          | 0%    | 30%    | 70%        | 0%   | sql/create_stage3_hive_tables.hql, scripts/stage4.sh, output/stage3_* files                                                                     |
| Stage 4: Superset charts                 | 10 EDA charts (Stage 2) + ML charts (metrics, training time, prediction distributions, hyperparameters, confusion matrix, feature summary) | 0%    | 70%    | 30%        | 0%   | Superset charts (slice IDs in dashboard)                                                                                                        |
| Stage 4: dashboard assembly              | Combine charts into single public Superset dashboard with descriptions                                                                     | 0%    | 100%   | 0%         | 0%   | published Superset dashboard                                                                                                                    |

| Project task                    | Description                                                                         | Artem | Arthur | Anastasiia | Alex | Deliverables                                             |
|---------------------------------|-------------------------------------------------------------------------------------|-------|--------|------------|------|----------------------------------------------------------|
| Repo structure, reproducibility | Repo init, .gitignore, main.sh glue, idempotent reruns                              | 20%   | 0%     | 0%         | 80%  | repo on GitHub, clean main.sh                            |
| Report and presentation         | Report sections, diagrams, presentation                                             | 0%    | 0%     | 0%         | 100% | report/report.pdf, presentation/presentation.pdf, slides |
| Pipeline cleanup and submission | Spark queue cleanup, empty-artefact removal, final commit + push, Moodle submission | 0%    | 5%     | 5%         | 90%  | clean cluster state, Moodle submission                   |

## Appendix. How to reproduce

On the cluster, as team22:

```
cd ~/project/bigdata-final-project  
bash main.sh
```

main.sh runs preprocess, stage1, stage2, stage3, stage4, postprocess, and pylint in order.  
All artefacts land in output/.